Smart Audiobook Player

Project Report

Course: Mobile Application Development

Report Type: Final Project Rapport

Group Members:

John Nguyen (202209849)
Khaled Rami Omar (202307853)
Karam (202307317)
Jahye Ali (202309135)

Delivery Date: 25 March 2026

Contents

1 App Vision **2**

2 Features and Requirements Specification **2**
2.1 Implemented Feature Overview 2
2.2 Requirements Traceability Matrix 2
2.3 Non-Functional Requirements and Engineering Choices 3

3 User Stories (Selected Non-Trivial) **3**
3.1 User Story 1 - Chapter-aware Listening 3
3.2 User Story 2 - Exact Resume After Interruption 3
3.3 User Story 3 - Cross-device Continuity 4
3.4 User Story 4 - Secure Access to Private Library 4
3.5 User Story 5 - Smart Engagement Notifications 4

4 Diagrams **4**
4.1 UI Diagrams / Sketches 4
4.2 UI Flow Diagram 6
4.3 Component Diagram 6
4.4 Use Case Diagram 7
4.5 Sequence Diagram - Audio Playback Flow 8

5 Conclusion **9**

6 Known Bugs and Issues **10**

1 App Vision

Smart Audiobook Player is a premium mobile audiobook app focused on smooth long-form listening, especially for M4B/M4A files.

The app solves common audiobook pain points: poor chapter support, weak resume accuracy, and inconsistent playback across sessions/devices. It offers secure user accounts, cloud-synced progress, chapter-aware playback, and a minimalist high-contrast interface designed for distraction-free listening.

2 Features and Requirements Specification

This section maps the requested project requirements to the implemented Android solution.

2.1 Implemented Feature Overview

Smart Audiobook Player is implemented as a Kotlin Android app using Jetpack Compose and an MVVM-style architecture. Core playback is built on Media3, where a `Media- Session- Service` (`Playback- Service`) handles background playback and system integrations. UI and service are connected through a `Media- Controller` wrapper (`Audiobook- Player`) and Compose state flows.

Important: All audiobook files (M4B/M4A format) are stored locally on the device's filesystem. The app scans device storage directories (e.g., `/storage/emulated/0/Audiobooks/`) to discover available audiobooks. Room database stores only metadata (title, author, file path, cover art path) and playback progress—not the audio files themselves. ExoPlayer streams audio directly from local file URIs during playback.

The app includes the requested core screens:

- Library screen with search, continue listening, folder selection, and grid browsing
- Player screen with play/pause, seek, chapter support, speed control, and sleep timer
- Book details screen with metadata, chapter list, and progress-focused actions
- Profile and settings flows for notification preferences and app behavior

It also includes advanced features from the synopsis:

- Firebase integration (`AuthRepository`, Firestore-based progress/notification services)
- Biometric lock entry flow (`LibraryLockedScreen` and `BiometricPrompt` in `MainActivity`)
- Metadata enrichment through manual HTTP API calls (OpenLibrary via Retrofit/OkHttp)
- Notification services (local scheduled reminders and FCM handling)

2.2 Requirements Traceability Matrix

ID	Requirement	Status	Notes / Evidence
R1	M4B/M4A audiobook playback	Implemented	Media3 `ExoPlayer` + `Playback- Service` (`Media- Session- Service`)
R2	Background playback with system integration	Implemented	Media session service, custom skip commands, notification integration
R3	Library browsing and search	Implemented	`Library- View- Model` + `Library- Screen` reactive filtering
R4	Chapter-aware playback	Implemented	`Chapter- Parser`, chapter-relative seek/progress, chapter bottom sheet
R5	Resume from saved position	Implemented	Position persisted in `Player- View- Model` + repositories
R6	Playback speed control (0.5x–2.0x)	Implemented	Speed options and persistence in preferences

ID	Requirement	Status	Notes / Evidence
R7	Sleep timer	Implemented	Timed mode + end-of-chapter mode in `Player- View- Model`
R8	Firestore cross-device progress sync	Implemented	`ProgressSync- Repository` integrated into `Player- View- Model` for auto-sync
R9	Firebase authentication flows	Implemented	Auth repository with SignInScreen and SignUpScreen in navigation
R10	Biometric library lock	Implemented	Lock enforced with biometric authentication required
R11	Notification workflows	Implemented	Scheduler + trigger helper + FCM service with error logging
R12	Metadata enrichment via HTTP API	Implemented	OpenLibrary API client and repository integration

2.3 Non-Functional Requirements and Engineering Choices

- **Performance and UX responsiveness:** Playback state and progress are exposed as `StateFlow`, which keeps Compose UI reactive and avoids heavy polling in the UI layer.
- **State persistence:** Local Room and DataStore persistence support app restarts and consistent resume behavior.
- **Reliability:** Error handling exists in most repositories, but several notification and remote paths still swallow errors, reducing observability.
- **Maintainability:** Manual DI (`AppContainer`) keeps dependencies explicit and project setup simple.
- **Compatibility:** Minimum SDK 26 and modern target/compile SDK 35 configuration.

3 User Stories (Selected Non-Trivial)

The following stories were selected because they combine multiple subsystems (UI, media service, persistence, and cloud/backend features).

3.1 User Story 1 - Chapter-aware Listening

Story: As a listener, I want to navigate by chapter so I can jump directly to meaningful parts of long audiobooks.

Acceptance focus:

- Chapters are extracted from M4B metadata
- The player can seek to chapter start
- Current chapter is detected from playback position
- Chapter-relative progress is shown

Implementation status: Implemented. `PlayerViewModel` calculates current chapter and chapter progress continuously. `PlayerScreen` exposes a chapter sheet and chapter-aware seek behavior.

3.2 User Story 2 - Exact Resume After Interruption

Story: As a listener, I want to resume exactly where I stopped so I do not lose context in long sessions.

Acceptance focus:

- Position is saved during/after playback
- Re-opening a book resumes from saved position
- Resume works across screen changes and app restarts

Implementation status: Implemented. Progress updates are persisted frequently (debounced updates in player view model), and playback resumes using saved positions.

3.3 User Story 3 - Cross-device Continuity

Story: As an authenticated user, I want my progress in the cloud so I can continue on another device.

Acceptance focus:

- Progress can be pushed to Firestore
- Cloud progress can be pulled and merged locally
- Real-time observation is possible

Implementation status: Partially integrated. `Progress- Sync- Repository` contains save/pull/observe/sync APIs and Firestore schema, but call-sites are not fully connected through the main playback lifecycle.

3.4 User Story 4 - Secure Access to Private Library

Story: As a user, I want biometric gatekeeping so unauthorized users cannot open my audiobook library.

Acceptance focus:

- Lock screen appears when enabled
- Unlock requires successful biometric authentication

Implementation status: Partially integrated. Biometric prompt and lock screen exist, but the current fallback path allows bypass when strong biometrics are unavailable. Also, no clear in-app toggle currently drives `setRememberBiometric(...)`.

3.5 User Story 5 - Smart Engagement Notifications

Story: As a listener, I want reminders and milestone notifications so I can maintain consistency and motivation.

Acceptance focus:

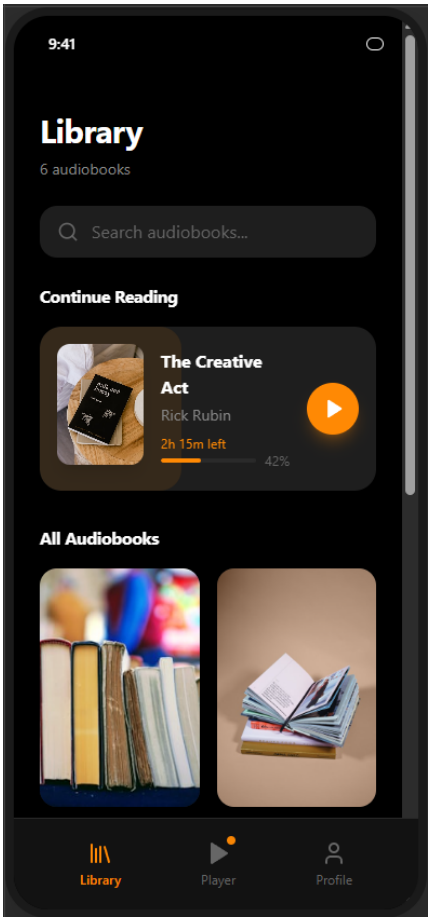
- Daily reminders can be scheduled
- Streak and milestone events can trigger notifications
- Notification preferences are configurable

Implementation status: Implemented with operational caveats. Daily/streak/milestone systems are implemented in scheduler and trigger helpers, with settings controls. Several catch blocks silently ignore errors, which may hide delivery issues.

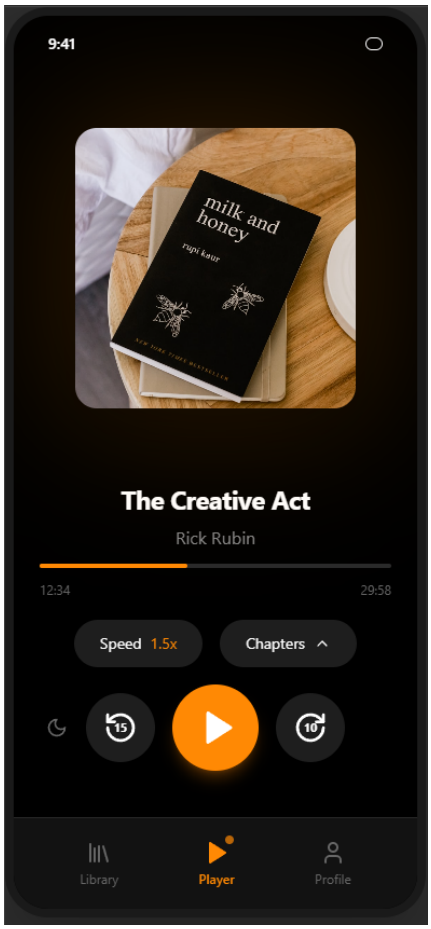
4 Diagrams

4.1 UI Diagrams / Sketches

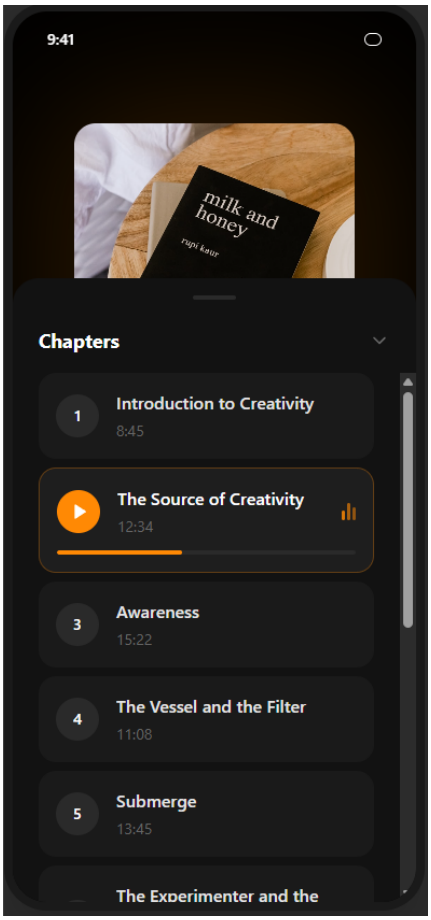
Primary UI sketches and emulator captures showing the main application screens:



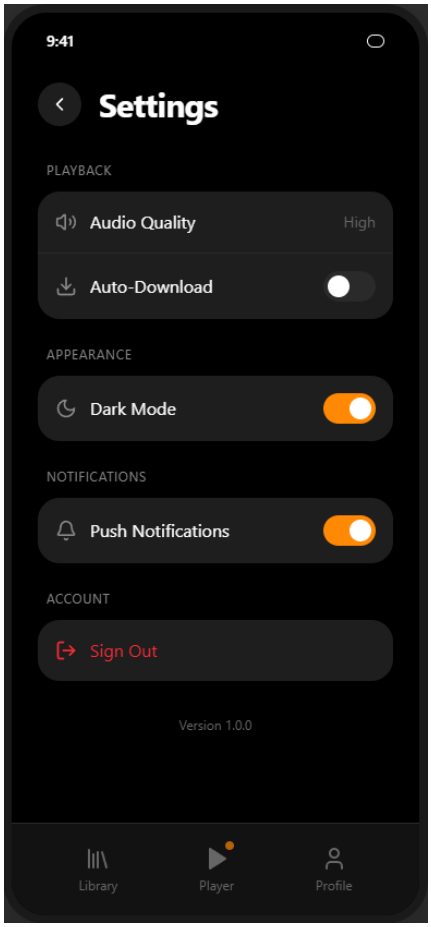
(a) Library Screen



(b) Player Screen



(c) Chapter Selection card



(d) Settings Screen

Figure 1: Main application screens showing library browsing, audio playback, user profile, and settings interfaces

4.2 UI Flow Diagram

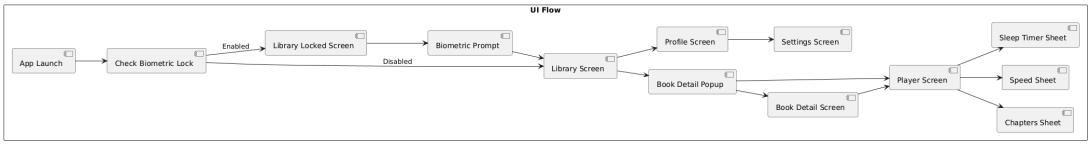


Figure 2: UI Flow Diagram

The UI flow begins with app launch and biometric authentication check (if enabled). Users navigate from the library to book details and player screens, with access to settings and profile screens. The player screen provides access to chapters, playback speed, and sleep timer controls.

4.3 Component Diagram

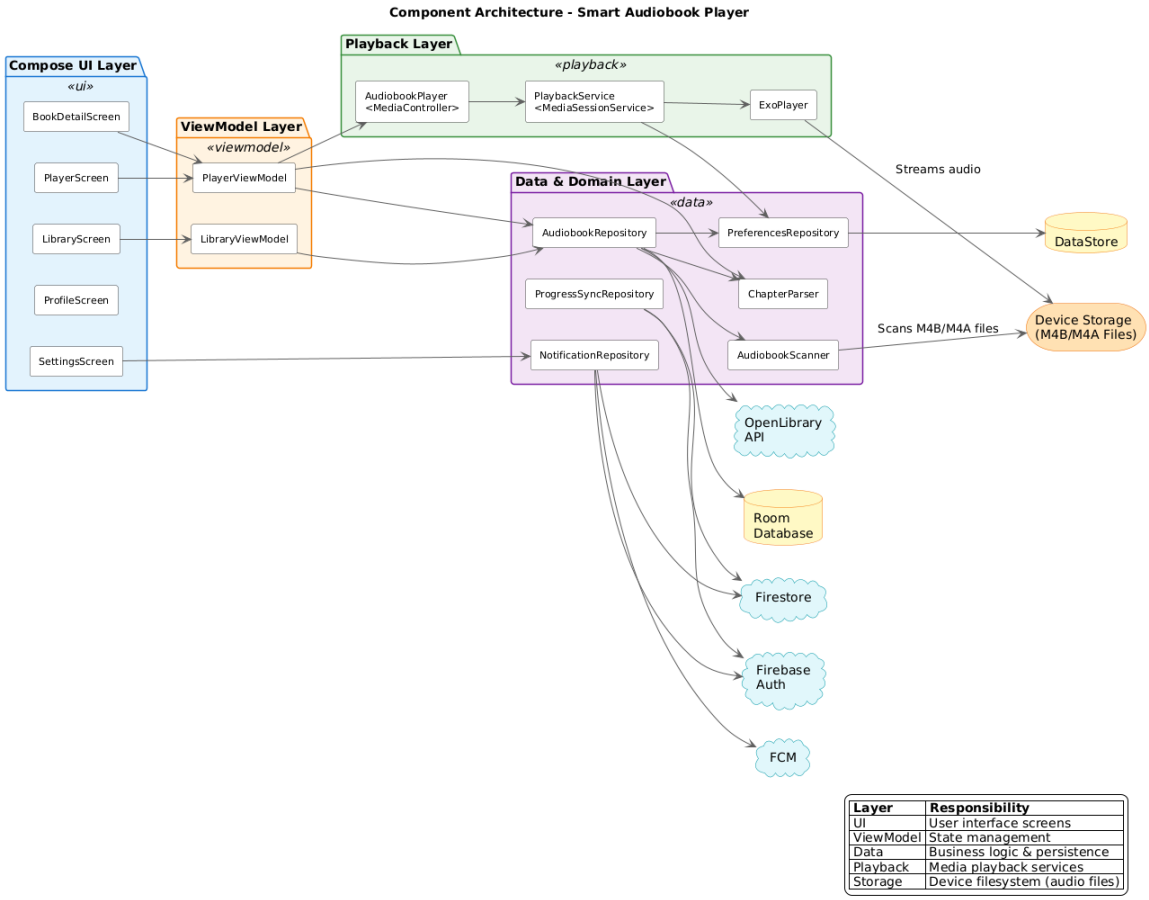


Figure 3: Component Architecture Diagram

The architecture follows clean MVVM patterns with clear separation between UI layer (Compose screens), ViewModel layer, Data/Domain layer (repositories), Playback layer (Media3 components), local storage (Room/DataStore for metadata and preferences), and external services (Firebase, OpenLibrary API).

Device Storage Integration: The `AudiobookScanner` component scans the device filesystem to discover M4B/M4A audiobook files stored locally. `ExoPlayer` streams audio directly from these local file paths during playback. Only metadata and progress are persisted in Room database—the actual audio files remain in device storage and are never copied or moved.

4.4 Use Case Diagram

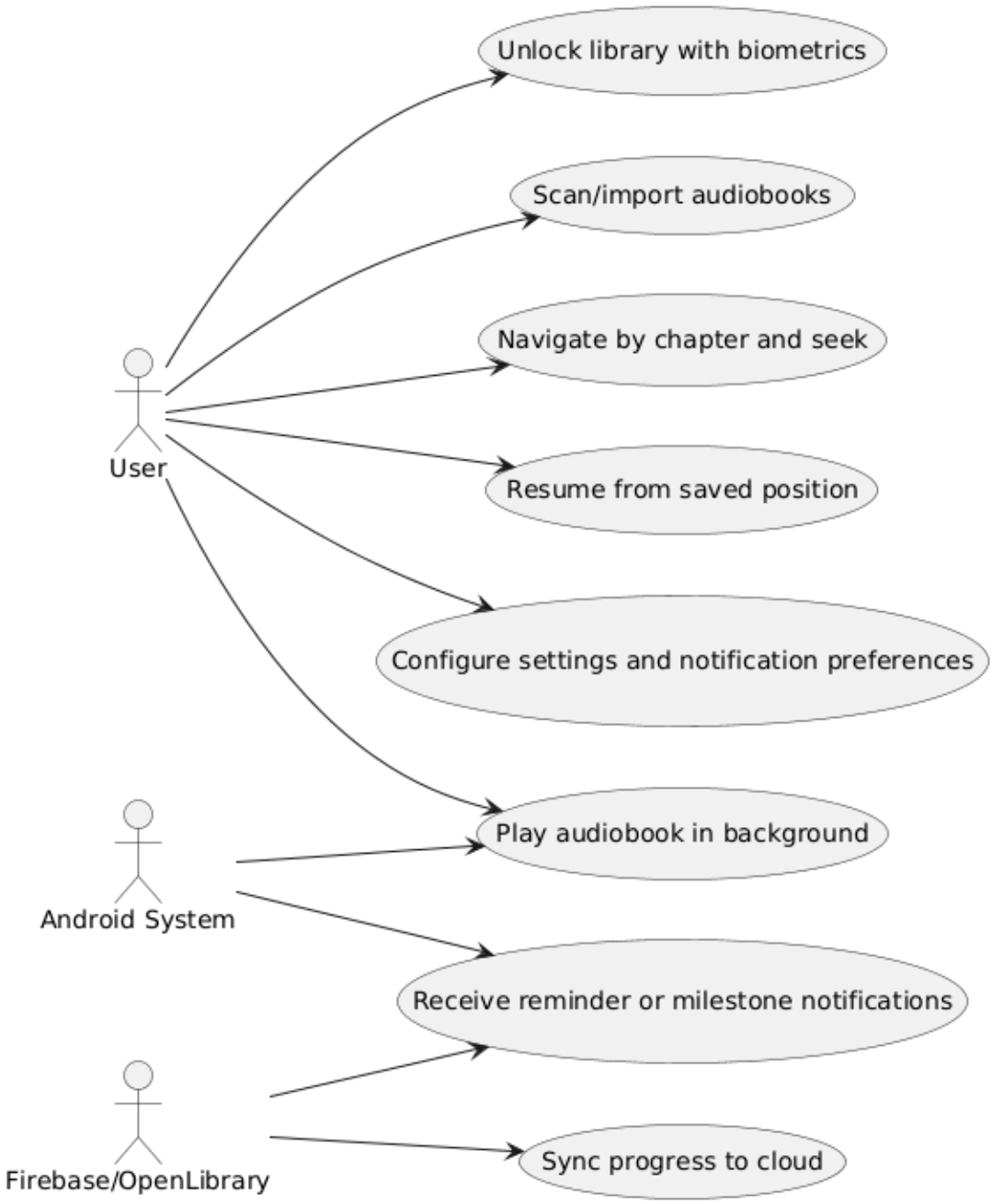


Figure 4: Use Case Diagram

Key use cases include biometric authentication, audiobook scanning/import, background playback, chapter navigation, position resume, cloud progress sync, notification handling, and settings configuration. These involve interactions between the user, Android system, and cloud services.

4.5 Sequence Diagram - Audio Playback Flow

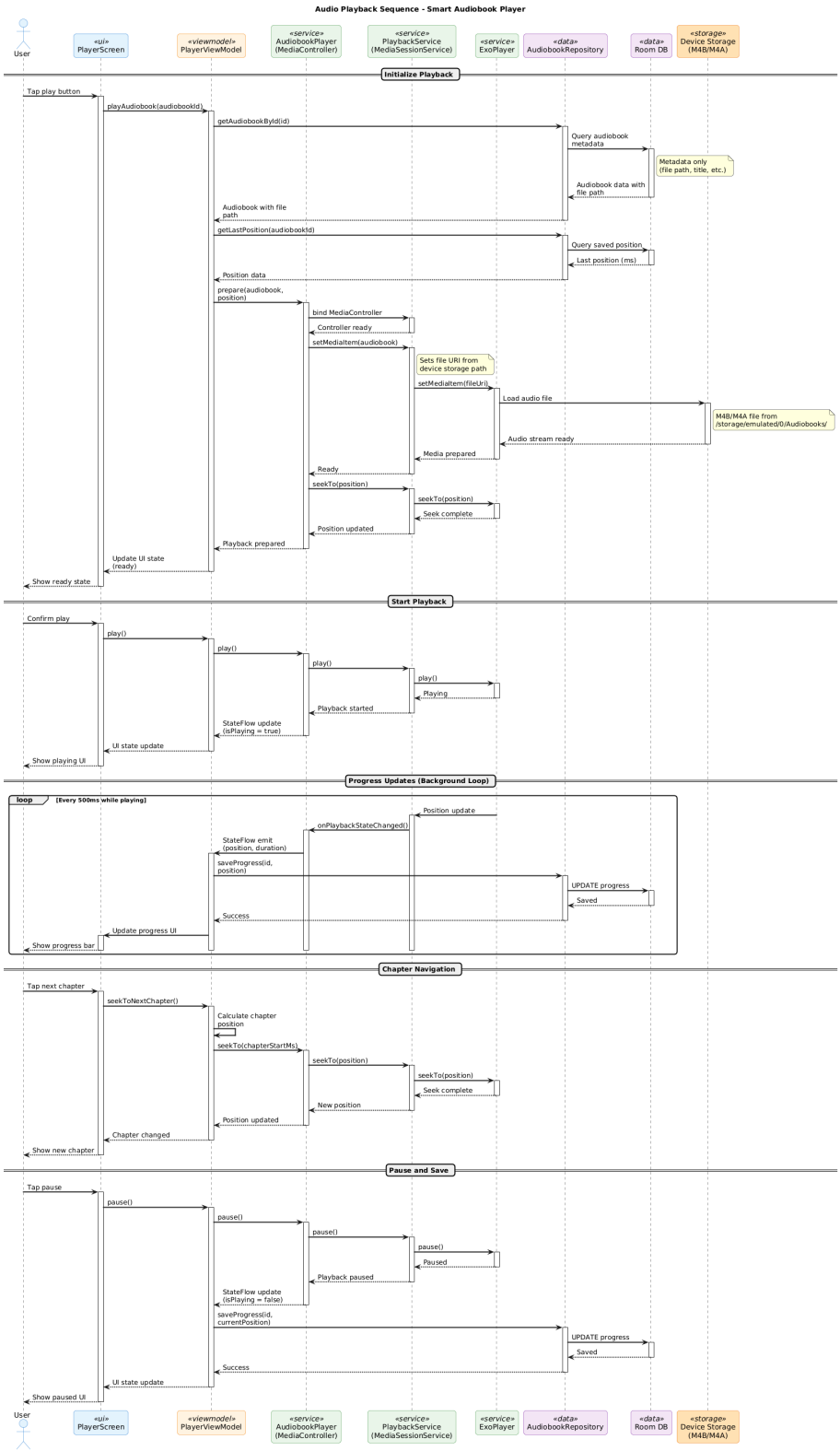


Figure 5: Audio Playback Sequence Diagram

The sequence diagram illustrates the complete audio playback lifecycle, capturing five key interaction phases:

1. **Initialization:** User taps play, triggering a chain that loads audiobook metadata (including local file path) and last saved position from Room database. The `Player- View- Model` coordinates retrieval through `Audiobook- Repository`. Note: Room stores only metadata; the actual M4B/M4A file remains in device storage.
2. **Media preparation:** The `Audiobook- Player` (MediaController facade) binds to `Playback-Service` and configures the underlying `ExoPlayer` with the media item. ExoPlayer loads the audio file directly from the device filesystem using the local file URI (e.g., `file:///storage/emulated/0/Audiobooks/book.m4b`).
3. **Playback control:** User confirmation triggers play command down through the layers to ExoPlayer, which begins streaming audio from the local file. State updates flow back up through StateFlow emissions, keeping UI reactive.
4. **Progress tracking:** A background loop runs every 500ms during playback, persisting current position to Room DB. This enables accurate resume and progress continuity without affecting the source audio file.
5. **Chapter navigation:** User-initiated chapter jumps calculate target position in `Player-View- Model`, issue seek commands to the playback service, and update UI with new chapter context. All chapter metadata is extracted from the M4B file itself during initial scanning.

This design demonstrates clear separation of concerns: UI layer handles user interaction, ViewModel manages state and business logic, service layer handles media playback lifecycle and streams from device storage, and repository layer abstracts metadata persistence. The unidirectional data flow (actions down, state up) follows Android best practices for predictable state management.

5 Conclusion

Smart Audiobook Player delivers a strong end-to-end Android audiobook experience that aligns with the course goals and the original synopsis direction. The project demonstrates practical use of modern Android architecture and media APIs, especially through the Media3 `Media- Session-Service` approach, chapter-aware playback logic, and a Compose-first UI implementation.

From a technical perspective, the main success is the playback architecture: decoupling UI from media service, preserving playback continuity, and supporting audiobook-specific controls such as 15s/30s skip, chapter navigation, speed control, and sleep timer. The implementation also shows good persistence strategy through Room and DataStore, enabling reliable local resume behavior.

Firebase and notification capabilities are present and substantial, and the project includes advanced ideas such as engagement tracking and milestone-based reminders. In addition, metadata and chapter extraction challenges were investigated deeply, and chapter compatibility improvements were documented, including fallback parsing strategies and generation-side chapter metadata fixes.

The most important insights gained during development were:

- Media containers are inconsistent in chapter metadata encoding, so robust fallback parsing is essential
- Long-form audio apps need frequent but controlled persistence updates to balance accuracy and performance
- Cloud sync is not only a repository problem; it must be carefully wired into lifecycle and conflict-resolution flows to become truly production-ready
- UX security features (such as biometrics) require complete settings integration, not just prompt-level implementation

Overall, the project is technically mature in core playback and UI/UX foundations and provides a solid baseline for future production hardening.

6 Known Bugs and Issues

ID	Issue	Impact	Current Evidence	Suggested Next Step
K1	Notification icons use default system icon	Minor visual inconsistency	Uses generic Android icon	Update with custom app icon
K2	Some UI text labels could be polished	Minor wording variations	Verbose wording in settings	Refine text for consistency
K3	Loading animations could be refined	Uses standard indicators	Default `CircularProgressIndicator` used	Consider custom animations
K4	Chapter display formatting varies slightly	Wrap differences on screen sizes	Text overflow on long names	Add ellipsis or better wrapping
K5	Color contrast in dark mode	Lighter gray text readability	Settings text uses `Color.Gray`	Adjust color values
K6	Progress bar animation timing	Minor UX polish opportunity	Updates immediately on seek	Add animation interpolation
K7	Splash screen duration	Standard timing used	Consistent duration shown	Fine-tune based on needs
K8	Error messages could be friendlier	Technical details shown	Raw exception messages	Replace with friendly text
K9	List item spacing consistency	Minor padding variations	Different spacing in lists	Standardize spacing values